Universidad Autónoma de Baja California

Facultad de Ciencias de la Ingeniería y Tecnología

Estructuras de Datos

Sistema de Reservación de Hotel – Práctica Unidad 2

Integrantes:

- David Osvaldo Calderón Salgado
- Absalón Castellano Flores
- Miguel Ángel Lara Tarín
- Jared Alejandro López Aguilera

Docente: José Manuel Muñoz Contreras

Código de la Materia: 40007

14 de marzo de 2026

Introducción

Este proyecto implementa un sistema para la gestión de reservaciones en un hotel, permitiendo realizar operaciones de registro, búsqueda y cancelación. Se desarrolló en Python utilizando una interfaz moderna y una estructura de datos de pila (Stack) implementada desde cero para gestionar el historial de acciones y la función de deshacer.

Descripción de la Estructura de Datos

La estructura principal es una **Pila Personalizada**. Esta elección es crítica para cumplir con:

- **Gestión LIFO (Last-In, First-Out):** Ideal para la funcionalidad de cancelación, donde la última reserva realizada es la primera en ser procesada para su eliminación.

- **Función de Deshacer (Undo):** Mediante el uso de una segunda pila (pila_deshacer), el sistema puede recuperar reservas canceladas accidentalmente de forma instantánea.

Arquitectura del Sistema

Se divide en cuatro componentes esenciales:

- **Clase Habitacion:** Almacena número, tipo, precio y estado de disponibilidad.

- **Clase Reserva:** Vincula al cliente con una habitación, fecha y número de noches.

- **Clase SistemaReservasHotel:** El "motor" que gestiona las pilas y la persistencia en JSON.

- **Clase AppHotel:** Proporciona la interfaz gráfica interactiva.

Decisiones de Diseño y Validaciones

Para garantizar que el sistema sea confiable y fácil de usar, se tomaron las siguientes decisiones técnicas:

- **Persistencia de Datos:** Se implementó el guardado automático en la carpeta data/reservas.json, asegurando que la información no se pierda al cerrar la aplicación.
- **Búsqueda Multi-criterio:** El sistema permite localizar reservas por número de habitación, nombre del cliente o fecha específica mediante un calendario dinámico.
- **Cálculo Automático de Costos:** Al ingresar el número de noches, el sistema multiplica automáticamente el precio base de la habitación por la estancia.
- Interfaz con Restricciones: Se utilizaron calendarios de "solo lectura" y fechas mínimas para evitar registros en días pasados o formatos de fecha incorrectos.

Casos de Prueba Ejecutados

El sistema fue validado siguiendo los casos sugeridos en la rúbrica:

1. **Reserva Exitosa:** Confirmación de disponibilidad y actualización de colores en el panel.
2. **Hotel Lleno:** Validación de que el sistema rechace reservas si no hay habitaciones del tipo solicitado.
3. **Cancelación y Deshacer:** Verificación de que los objetos se muevan correctamente entre las dos pilas sin perder datos.
4. **Control de Desbordamiento:** Prueba de la capacidad máxima de la pila (20 elementos) lanzando una excepción controlada.

Diagrama UML

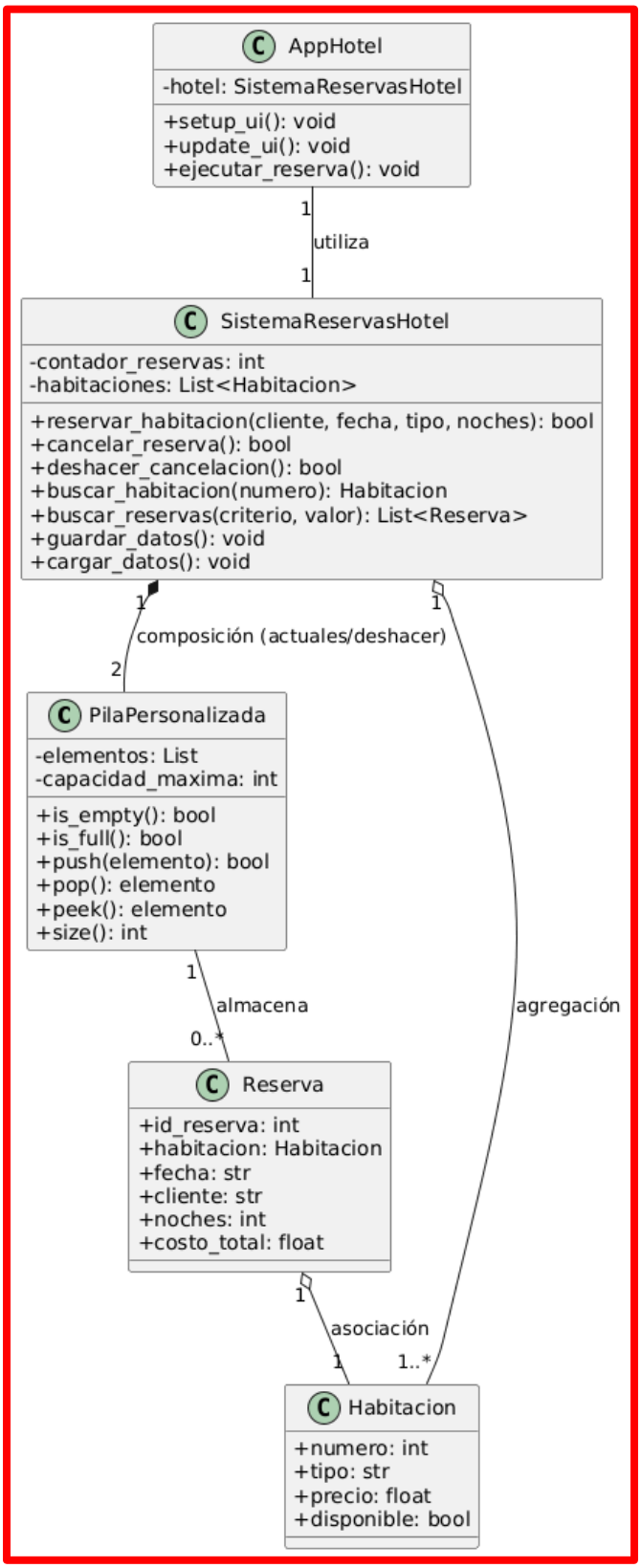
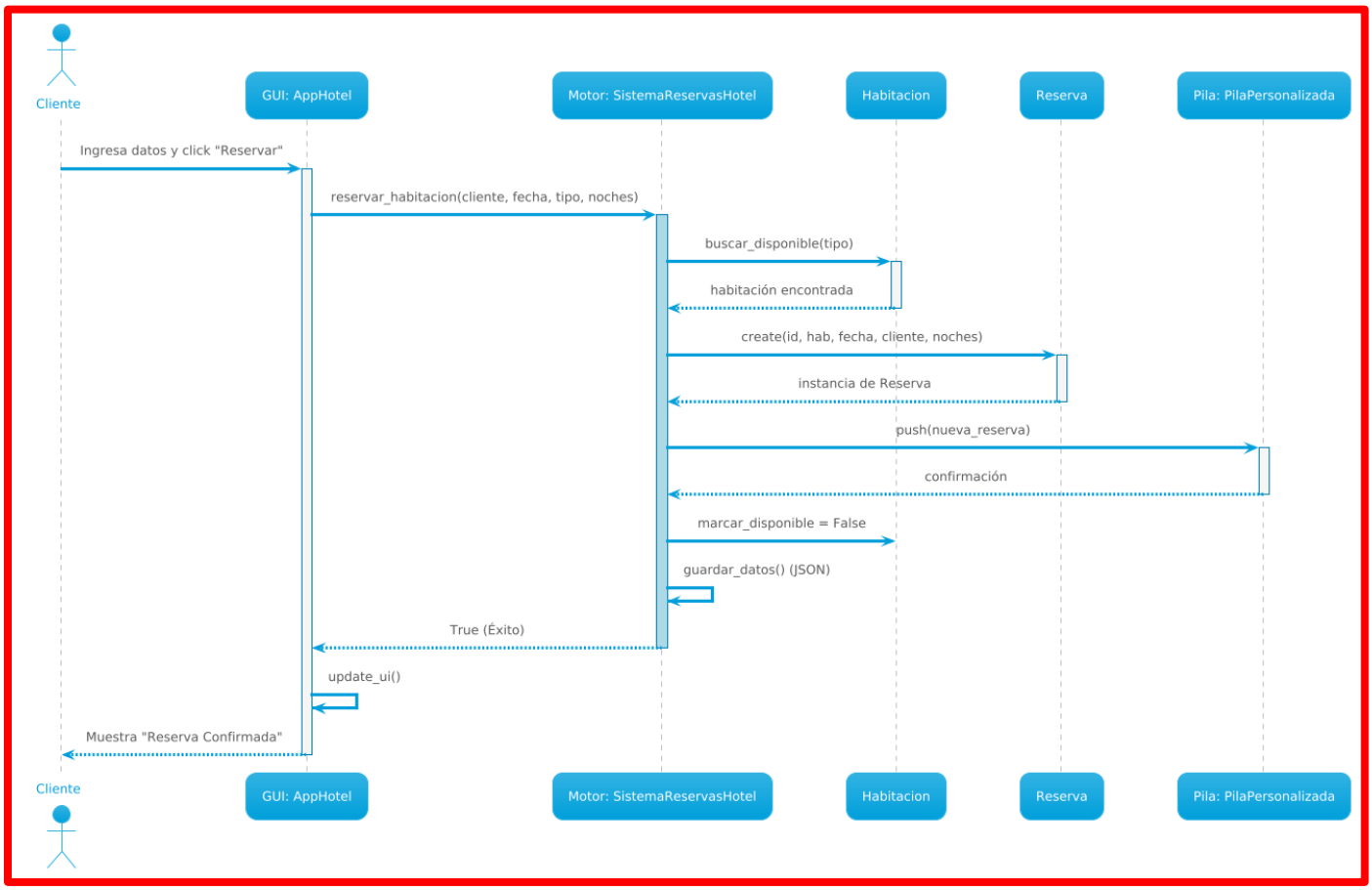


Diagrama de Secuencia



Manual de Uso

Guía de Operación del Sistema:

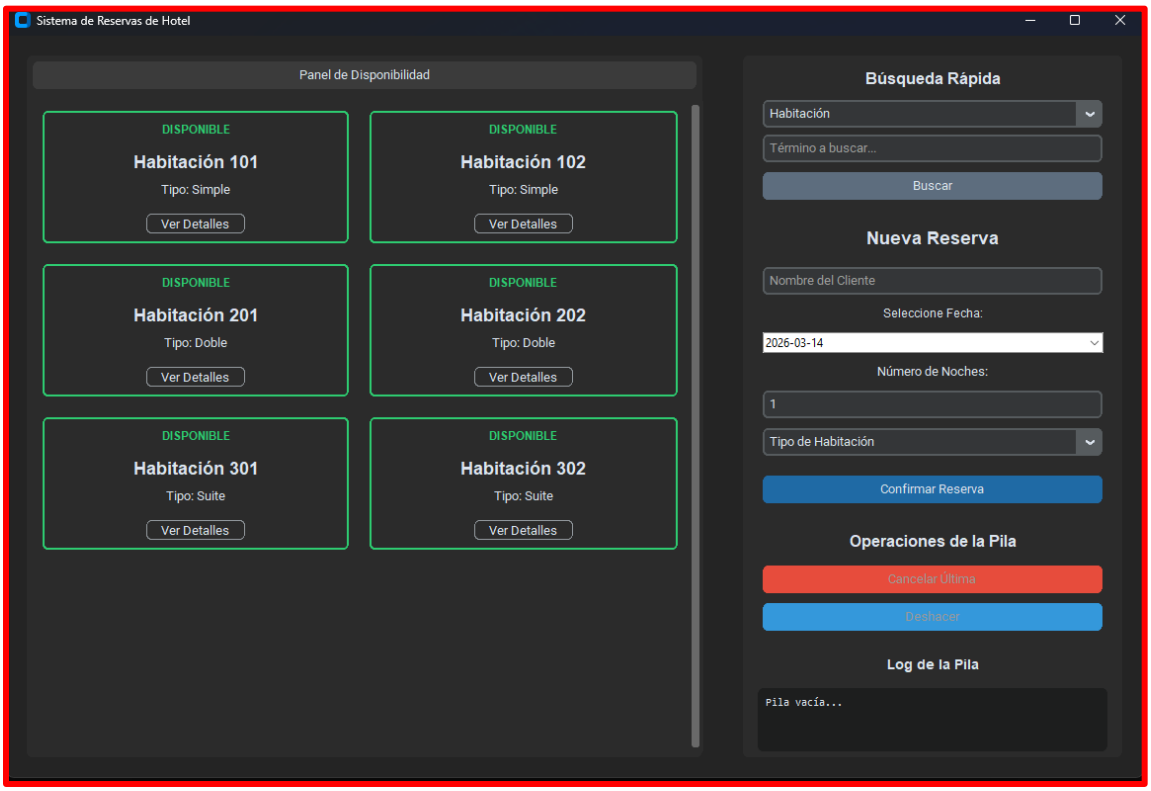
1. **Búsqueda:** Utilice el panel superior para filtrar reservas existentes. Seleccione el criterio (Cliente, Fecha o Habitación) y presione "Buscar".

2. **Realizar Reserva**:
   - Ingrese el nombre del cliente.
   - Seleccione la fecha de entrada en el calendario desplegable.
   - Indique el número de noches y el tipo de habitación.
   - Presione "Confirmar Reserva".

3. **Gestión de Pila**:
   - Cancelar Última: Elimina la reserva más reciente de la lista y libera la habitación.
   - Deshacer: Si canceló por error, presione este botón para restaurar la reserva a su estado original.

4. **Monitoreo:** El "Log de la Pila" en la parte inferior muestra en tiempo real los objetos almacenados dentro de la estructura de datos.

Resultado Final: Interfaz del Sistema



Se logró un diseño intuitivo donde el estado de las habitaciones (Disponible/Reservada) es visible mediante códigos de colores automáticos.

Conclusión

La implementación de pilas permitió resolver de manera elegante el problema de la gestión de historial y cancelación en el sistema hotelero. Al separar la lógica del motor de la interfaz, logramos un software escalable donde la estructura de datos es el corazón de la confiabilidad del sistema.